



Multi-generation multi-criteria feature construction using Genetic Programming

Jianbin Ma^{a,b,*}, Xiaoying Gao^c, Ying Li^d

^a College of Information Science and Technology, Hebei Agricultural University, Baoding, 071001, China

^b Hebei Key Laboratory of Agricultural Big Data, Baoding, 071001, China

^c School of Engineering and Computer Science, Victoria University of Wellington, Wellington, 6140, New Zealand

^d College of Economics and Management, Hebei Agricultural University, Baoding, 071001, China

ARTICLE INFO

Keywords:

Feature construction
Genetic programming
Overfitting
Multi-generation
Multi-criteria

ABSTRACT

The purpose of feature construction is to create new high level features from the original features. When Genetic Programming (GP) is applied to wrapper-based feature construction, especially when the samples size is small, GP generally overfits the training set and generalizes poorly with the deepening of evolution. Overfitting has attracted wide attention in some classification models, however, it is not commonly studied in the field of feature construction. In this paper, a Multi-Generation feature construction method (MG) is developed to preserve the solutions produced by multiple generations of GP. A Multi-Criteria feature construction method (MC) is introduced to use a multi-criteria evaluation function to evaluate GP individuals. Combining the above two methods, a Multi-Generation Multi-Criteria feature construction method (MGMC) is proposed. Experiments on fourteen datasets show that the proposed MG and MC methods can improve the classification performance and overcome overfitting problems of traditional feature construction methods in most cases. The combined MGMC method further improves the classification performance and achieves the best results.

1. Introduction

The quality of features is one of the key factors that affects classification performance [1,2]. In some practical applications, original features do not have sufficient discriminative ability. Feature construction is applied to create new high level features from original features and to improve the classification performance and reduce the dimensions of features [3]. Genetic Programming (GP) is an effective evolutionary computation algorithm due to its global search ability [4–7]. GP can be represented as a tree-like logical and mathematical expression, so it is suitable for solving feature construction tasks [8–10].

According to different evaluation criteria, feature construction can be divided into two approaches: filter-based approach and wrapper-based approach. The filter-based approach uses intrinsic measures such as Information Gain (IG) [11], Information Gain Ratio (IGR) [12] and Fisher criterion [13] to evaluate a constructed feature. The wrapper-based approach uses the classification performance of a learning algorithm to evaluate a constructed feature. Compared with other feature transformation techniques such as principal component analysis (PCA) [14] and auto-encoders [15], feature construction methods using GP have better readability and interpretability. In addition, GP can automatically discover the hidden relations between features and the constructed features can lead to good classification results [16].

In the wrapper-based feature construction approach using GP, the GP evolves a fittest solution that fits the training set according to the fitness function. Then, the evolved solution is applied to the testing set to predict the class labels. However, the solutions sometimes overfit the training sets and show poor predictions on testing sets, especially in the applications with small samples. In addition, the degree of overfitting depends on the depth of the search [17]. If GP works too hard and searches for the very best solution from the training set, GP may fit noise data and memorize various peculiarities of the training set rather than finding a general predictive rule [18].

Overfitting has attracted much attention in classifiers such as neural networks [19] and decision trees [20]. Loughrey [17] studied the overfitting problems of wrapper-based feature selection methods using Genetic Algorithms and found that overfitting was related to the depth of a search. However, not much research focuses GP-based feature construction methods which search for the fittest solution by iterating the evolution process generation by generation. The key to avoiding the overfitting of feature construction is to know which generation can produce a good solution in the process of GP evolution. Early stopping [17,21–23] is a common method to solve the overfitting problem. The early stopping needs to separate an evaluation set from the training

* Corresponding author at: College of Information Science and Technology, Hebei Agricultural University, Baoding, 071001, China.

E-mail addresses: majianbin@hebau.edu.cn (J. Ma), Xiaoying.Gao@ecs.vuw.ac.nz (X. Gao), sxliying@hebau.edu.cn (Y. Li).

set and needs to stop training when classification performance on the evaluation set starts to deteriorate. Since the evaluation set is separated from the training set, the training set is even smaller which may affect the generalization ability of the constructed feature [17]. Moreover, it is difficult to find the stopping point because the classification performance on unseen data oscillates within a certain generation range which is shown in Fig. 2. In this paper, we propose a Multi-Generation feature construction method (MG) to avoid overfitting problems, which preserves multiple solutions generated by multiple generation of GP.

In general, wrapper-based feature construction methods use the classification performance of a learning algorithm as the fitness function. There are two drawbacks in the wrapper-based methods. (1) During the evolution of GP, the fitness of individuals will have the same value for many individuals. Thus, it is impossible to know which individual has the better discriminative ability only by the evaluation criterion of classification performance. (2) GP will easily memorize the characteristics of the training set only evaluated by the classification performance of a learning algorithm, which leads to overfitting. Intrinsic characteristic measures such as Information Gain Ratio (IGR) and Information Gain (IG) are also effective methods to evaluate the effectiveness of a constructed feature. Adding the second criterion can distinguish multiple best solutions obtained by only the wrapper-based method better. Therefore, in this paper, a Multi-Criteria feature construction method (MC) which preserves the fittest solution evaluated by a multi-criteria evaluation function that combines two evaluation criteria consisting of classification performance and IGR is proposed. In addition, combining MG and MC, we propose a Multi-Generation Multi-Criteria feature construction method (MGMC) and verify whether this method can achieve better classification performance than MG, MC and traditional single feature construction methods.

1.1. Research objectives

The overall goal of this paper is to investigate feature construction methods to overcome overfitting problems. The following four objectives will be investigated.

Objective 1: Investigate the traditional single feature construction method (S) and verify whether this method can result in overfitting problems.

Objective 2: Propose a Multi-Generation feature construction method (MG) and verify whether this method can improve the classification performance of S, and overcome overfitting of S.

Objective 3: Propose a Multi-Criteria feature construction method (MC) and verify whether this method can also overcome overfitting of S.

Objective 4: Propose a Multi-Generation Multi-Criteria feature construction method (MGMC) and verify whether this method can achieve better classification performance than S, MG and MC.

1.2. Organization

The remainder of the paper is organized as follows. Section 2 provides background information. Section 3 describes the proposed multi-generation multi-criteria feature construction method. Section 4 details our experimental settings and Section 5 presents the experimental results and discussions. Section 6 provides conclusions and future work.

2. Related work

2.1. Overfitting

In statistics and machine learning, some classifiers are easy to overfit due to their particular structures. Early stopping [21,22] is a common method to address overfitting which stops the training as soon as classification performance on a validation set starts to get worse.

Some methods, including simplifying neural networks by weight penalties [24] and dropout used to drop units from neural networks during training [19] are utilized to prevent overfitting of neural networks. Pruning methods [20] are often used to eliminate overfitting problems of decision trees.

Overfitting also occurs in regression models. Regularization [18,25] helps to prevent overfitting by constraining the model and reducing the impact of the noise. Validation and cross-validation [26,27] are useful techniques for estimating the error of unseen data and are used for model selection. For GP-based symbolic regression models [28–30], methods for reducing or controlling the model complexity have been proposed to avoid overfitting problems.

In recent years, some studies [30–32] investigated methods to quantify overfitting in order to evaluate the ability to generalize the model. Some researchers [33,34] used dimensionality reduction techniques to reduce the overfitting. Most studies focused on overfitting problems in classification and regression. Loughrey [17] used early stopping methods to solve overfitting problems on feature selection. Methods for avoiding overfitting are not commonly used for producing models with GP. Gonçalves et al. (2017) [23] used semantic criteria to make GP decide when to stop the search in classification. Batista and Silva (2020) [35] investigated that mixed datasets could affect the generalization of trained models on GP-based constructed features. Existing methods for avoiding overfitting problems produce a single solution. In this paper, we proposed a GP-based multi-generation feature construction method which constructs multiple features produced by multiple generations of GP. We do not rely on finding the early stopping point like other methods. Moreover, we proposed a multi-criteria evaluation method to prevent GP from converging easily and to overcome overfitting.

2.2. Feature construction approaches using GP

2.2.1. Filter versus wrapper

Filter-based feature construction approaches use intrinsic measures such as information gain (IG) [11], information gain ratio (IGR) [12] and Fisher criterion [13,36]. Otero et al. [12] proposed a feature construction method that used IGR as the fitness function. Muharram et al. [37] proposed a feature construction method that used four fitness functions including information gain, gini index, chisquare, and a combination of information gain and gini index. Guo et al. [36,38,39] proposed filter-based feature construction approaches based on the Fisher criterion or a modified Fisher criterion. Neshatian et al. (2012) [8] proposed a filter-based multiple feature construction method using GP. A feature is constructed for each class by running an independent GP once, and an entropy based fitness function is utilized to maximize the purity of the boundary area between classes. Liao et al. (2021) [40] used a binary decomposition-based GP method to construct new features to solve the multiclass classification task. Pearson's correlation coefficient was used as the fitness function.

Wrapper-based feature construction approaches evaluated constructed features by directly using the classification performance of a learning algorithm so that the approaches are classifier dependent. In general, each GP tree can be transformed to a single constructed feature [41]. Due to the flexible representation methods of GP, GP trees can be used to create more constructed features with stronger discriminative ability than a single constructed feature. Many wrapper-based multiple feature construction approaches [13,42–44] were proposed to improve the classification performance.

2.2.2. Single constructed feature versus multiple constructed features

In general, GP outputs the fittest individual and has been used to construct a single feature. Since using a single constructed feature is not enough for giving an acceptable classification performance, the constructed feature is often added to original features [8] and the augmented features are used as the feature set. Example include the three

filter-based feature construction methods mentioned above [11,12,39], which adds a single constructed feature into original features.

Some multiple feature construction methods were proposed to improve the classification performance of single feature construction methods. Neshatian et al. [8] is one example of a multiple feature construction approach. Cooperative coevolution strategy [42,45–47] that creates multiple concurrent populations is another method to construct multiple features. Ahmed et al. [13] proposed a multiple feature construction method that constructs multiple features from all possible subtrees of the best individual. An archive-based feature construction method proposed by Krawiec et al. [43] that stored excellent individuals during GP runs were used to construct multiple features. Ma & Teng [48] preserved top excellent individuals during a single GP run, and these individuals were transformed to multiple features. Tran et al. [49,50] used a multiple-tree representation method to construct multiple features, and the leaf nodes of multiple trees were also selected as features.

To the best of our knowledge, few studies have investigated whether there are overfitting problems in current feature construction methods and how we can deal with these overfitting problems. So, in this paper, our research focuses on (1) investigating whether there is overfitting in traditional single feature construction method (S), (2) verifying whether our proposed multi-generation feature construction method (MG) and multi-criteria feature construction method (MC) can overcome overfitting of S, (3) verifying whether our proposed multi-generation multi-criteria feature construction method (MGMC) can improve the classification performance comparing with S, MG and MC.

3. Methodology

3.1. Multi-generation feature construction method

In GP-based feature construction methods, overfitting occurs when GP exceeds a certain generation. However, it is difficult to estimate which generation produces the fittest individual. As shown in Fig. 2, the classification performance on the testing set oscillates severely with the increase of generations. The fittest individual generated from the training set may not necessarily achieve the best predictive performance on the testing set. In this paper, a multi-generation feature construction method is proposed, which preserves the best individuals produced by each generation of GP. We verify whether it can overcome overfitting problems of the single feature construction method and increase discriminative ability of constructed features.

Suppose the best individuals produced by all generations are represented as $I_a = \{i_1^a, i_2^a, \dots, i_m^a\}$, where i_k^a is the best individual produced by k th generation, and m is the number of generations of GP. If all the m individuals are taken as constructed features, there will be a large number of redundant features. In fact, with the GP running, the algorithm converges gradually. The best individuals produced by each generation will have the same fitness. As shown in Fig. 2, on most datasets, the fitness value of the best individual rises gradually with the increase of the generation and does not change any more when the generation exceeds one certain value. Therefore, we remove the redundant individuals that have the same fitness. The remaining individuals have different fitness, and these individuals are represented by $I_d = \{i_1^d, i_2^d, \dots, i_n^d\}$, where n is the number of different individuals.

3.2. Multi-criteria feature construction method

Wrapper-based feature construction methods evaluate the fitness of a constructed feature according to the classification performance of a learning algorithm. In many cases, the solutions generated by GP have the same fitness if they are evaluated only by classification performance, but they have different discriminative ability. GP will easily memorize the characteristics of a training set if it is evaluated only by the classification performance of a learning algorithm so that

the solution generated by GP overfits the training set easily with the growth of the depth of search. In this paper, a multi-criteria feature construction method is proposed, which preserves the fittest solution evaluated by a multi-criteria evaluation function that combines the classification performance and the Information Gain Ratio (IGR).

The terminal set of a GP individual consists of the original features. The values of original features are inputted into GP individuals. The output of a GP individual is the value of the new constructed feature. Then, an original dataset is transformed to another dataset according to the new constructed features. 10-fold cross-validation is used to obtain the wrapper fitness of the new constructed features in the following Eqs. (1) and (3).

The error rate which is shown in Eq. (1) is chosen to evaluate the classification performance of a learning algorithm, so the lower the better.

$$evaluation_1 = ErrorRate = \frac{FP + FN}{TP + TN + FP + FN} \quad (1)$$

where *ErrorRate* is the error rate of a constructed feature for classification. *TP*, *TN*, *FP* and *FN* denote true positives, true negatives, false positives and false negatives of the classification results, respectively.

IGR is chosen as another evaluation criterion which is shown in Eq. (2). It is adjusted so the lower the better.

$$evaluation_2 = (1 - IGR) \quad (2)$$

where IGR is the information gain ratio of a constructed feature.

The multi-criteria evaluation function that combines the above two evaluation criteria is shown in Eq. (3).

$$evaluation_3 = \alpha * evaluation_1 + (1 - \alpha) * evaluation_2 \\ = \alpha * ErrorRate + (1 - \alpha) * (1 - IGR) \quad (3)$$

where α is the relative importance of *evaluation_1* and *evaluation_2*, and $\alpha \in [0, 1]$. If $\alpha > 0.5$, *evaluation_1* is more important and weighted higher than *evaluation_2*.

Our proposed feature construction methods are all wrapper-based, so the classification performance in Eq. (1) is used as the fitness function of GP to evaluate individuals. The multi-criteria evaluation function in Eq. (3) is used to preserve more effective solutions than using classification performance alone. Evaluation functions in Eqs. (1) and (3) are used in different ways, so we have different variants of the algorithms which are shown in the next subsection.

3.3. Proposed feature construction methods

In this paper, we compare four GP-based feature construction approaches as follows.

Single feature construction approach (S): The algorithm evaluates individuals according to the fitness function in Eq. (1) and outputs the fittest individual.

Multi-criteria single feature construction method (MC): The algorithm evaluates individuals according to the fitness function in Eq. (1) and preserves the best individual according to the multi-criteria evaluation function in Eq. (3).

Multi-generation feature construction approach (MG): The algorithm preserves multiple individuals produced by multiple generations of GP according to the fitness function in Eq. (1).

Multi-generation multi-criteria feature construction approach (MGMC): The algorithm preserves multiple individuals produced by multiple generations of GP according to the multi-criteria evaluation function in Eq. (3). Please note that this algorithm still uses Eq. (1) as the fitness function for evaluating each individual. The pseudo code of this algorithm is shown in Algorithm 1.

Algorithm 1: Pseudo-Code of MGMC algorithm

Input: D : a training set
Output: The constructed features

- 1 Randomly initialize the individuals in the population;
- 2 **for** $k=1$ to m (*Maximum Generation*) **do**
- 3 **for** $j=1$ to *Population Size* **do**
- 4 Evaluate individuals on Training set according to the fitness function in Equation (1);
- 5 Update the best individual according to the evaluation function in Equation (3) and its corresponding evaluation values;
- 6 **end**
- 7 Preserve the best individual i_k and corresponding evaluation values for this generation;
- 8 Select individuals from the population using the Tournament method;
- 9 Reproduction, Crossover and Mutation on individuals in the population;
- 10 **end**
- 11 Produce the best individuals for all generations
 $I_a = \{i_1^a, i_2^a, \dots, i_m^a\}$ and their corresponding evaluation value
 $F_a = \{F_1, F_2, \dots, F_m\}$;
- 12 Removes redundant individuals that have the same fitness and obtains the individuals $I_d = \{i_1^d, i_2^d, \dots, i_n^d\}$;
- 13 **Output** the constructed features

Table 1
Description of the fourteen datasets.

Dataset	#Features	#Instances	#Classes	# α
Glass	9	214	6	0.8
Wine	13	178	3	0.8
Sonar	60	208	2	0.1
Iris	4	150	3	0.6
Thyroid disease	5	215	3	0.4
Liver-disorders	6	345	2	0.8
Wdbc	30	569	2	0.5
WBC	9	683	2	0.8
Colon	2000	62	2	0.8
Leukemia	7129	72	2	0.8
Lymphoma	4026	62	3	0.8
SRBCT	2308	83	4	0.8
MLL	12582	72	3	0.9
Ovarian	15154	253	2	0.9

α shows the best parameter value α .

4. Experiments

Fourteen datasets involving classification problems in Table 1 are used to evaluate the performance of our proposed feature construction methods. The first eight datasets are collected from the UCI machine learning repository [51]. The last six datasets are microarray datasets collected from <http://csse.szu.edu.cn/staff/zhuzx/datasets.html> [52]. These datasets have different number of features, instances and classes, and are used as the representative examples for the classification problems. The first three datasets are in the physical domain, and are used to predict glass, wine or sonar signal respectively. The remaining datasets belong to the life domain which is used to predict a disease. In addition, these datasets have a small number of samples and are prone to overfitting problems, which are more suitable to verify our approaches.

In order to get more convincing experimental results, each dataset is randomly divided into 30 different training sets and testing sets using 30 different random seeds [10], with 70% as the training set and 30% as the testing set [53]. All the GP based feature construction algorithms have been run 30 times on 30 different training sets and testing sets independently to avoid the effect of stochastic characteristics of GP

Table 2
Parameter setting of GP.

Parameters	Parameter value
Function set	+, −, ×, % (protected division)
Terminal set	Original features
Population size	500
Number of generations	50
Initialization	Ramped half-and-half
Mutation method	Random subtree creation
Selection method	Tournament method (size = 7)
Maximum tree depth	17
Mutation probability	10%
Cross-over probability	90%
Reproduction probability	10%

and dataset partition. Training sets are used to construct new features. Testing sets are used to evaluate the feature construction methods. The classification accuracy is used to evaluate the experimental results. In Sections 5.1 to 5.5, the K-nearest Neighbour (KNN, $K = 5$) learning algorithm is used to evaluate the classification performance of the proposed methods.

The ECJ library [54] is used to run GP. The parameter settings of GP are shown in Table 2, and they are set based on other common GP-based feature construction methods [8,10,48]. The function set of GP contains four arithmetic operators (+, −, ×, %). The protected division (%) results in zero when dividing by zero. The population size is set to 500. The number of generations is set to 50. The ramped half-and-half method is used for generating programs in the initial population. Tournament selection is used with a tournament size of seven, which is consistent with the parameter settings in other literature [8,49,55]. The maximum tree depth is 17 to restrict the size of the GP program. Mutation probability and crossover probability are 10% and 90% [16, 48], respectively, which balance exploration and exploitation during the evolution. Note that fine tuning GP's parameters is outside the scope of our work; this paper focuses on investigating methods to avoid overfitting problems of feature construction.

Different datasets have preferences for a different parameter value α in Eq. (3). The best parameter value α for different datasets is shown in the fifth column of Table 1. The impact of parameter α on experimental results will be discussed in Section 5.4. In the following experiments, four algorithms in Section 3.3 will be tested on fourteen datasets separately. A total of (4algorithms * 14datasets) * 30 GP = 1680 GP is run. In order to adjust α and further analyze its effects, we have run GP several times.

5. Experimental results and discussions

5.1. The analysis of feature construction

The GP-based single feature construction method aims to evolve a fittest solution that is generated from training data. The solution transforms the original features into a new feature and the new feature is used for classification.

Fig. 1 demonstrates the discriminative ability of a constructed feature on Sonar, iris, Wdbc and WBC datasets. In order to show the significant effects, we jitter one-dimensional data into a two-dimensional space, where the X -axis value of a sample is the value of the constructed feature and the Y -axis value is a random number. As shown in Fig. 1, the feature constructed by GP has a good discriminative ability between classes on iris, Wdbc and WBC datasets, while the feature constructed by GP on the Sonar dataset cannot distinguish all of the positive samples and negative samples. Although feature construction can improve the classification performance of the Sonar dataset compared with the original features, the classification performance using original features itself is very poor. Feature construction has not improved the classification performance significantly.

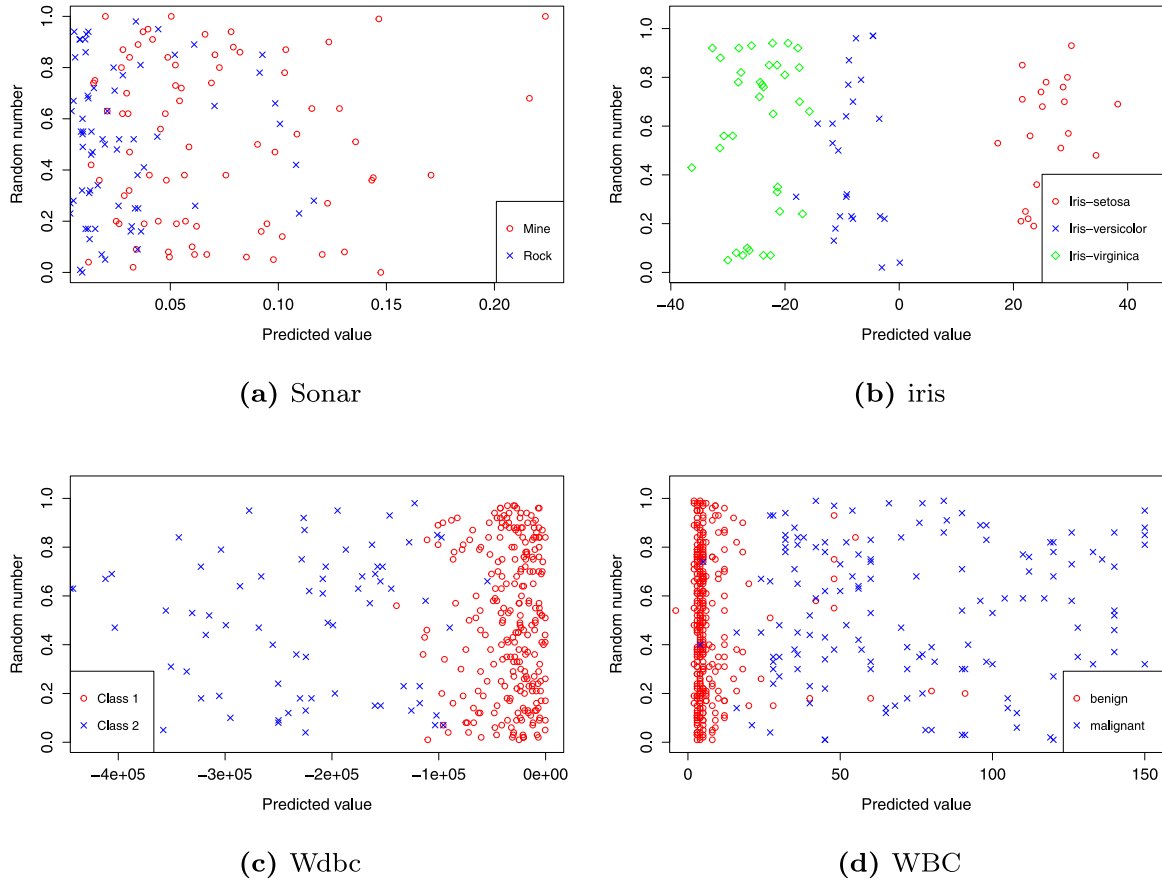


Fig. 1. The demonstration of discriminative ability of the constructed feature on Sonar, iris, Wdbc and WBC datasets.

As shown in Fig. 1, the distribution of the new sample points in each class is uneven. For example, for datasets Wdbc and WBC, the sample points of one class is more concentrated, while those of the other class is more dispersed. With the increase of GP iteration, the sample points will be more and more concentrated. Therefore, the constructed features maybe perform very well on the training set, but they are less effective on the testing set.

5.2. Reasons for overfitting

Fig. 2 shows the overfitting phenomena of the single feature construction method (S) in six datasets. As shown in Fig. 2, the accuracy on training sets increases gradually with the increase of generations. After reaching a certain generation, the accuracy on training sets is stable and does not change any more. In comparison, the accuracy on testing sets increases at first, but the general trend is downward after a certain generation. As shown in Table 1, the number of instances in the six datasets is small. Different fold partitions of k-fold cross-validation has a great impact on the classification performance. It shows that the accuracy of testing sets fluctuates.

To show some more evidence about overfitting in feature construction, taking the Sonar dataset as an example, the predictive effect of constructed features on different generations is shown in Fig. 3. Figs. 3(a) and 3(b) show the discriminative ability of constructed features for the 6th generation on the training set and testing set respectively. Figs. 3(c) and 3(d) show the discriminative ability of constructed features for the 50th generation on the training set and testing set respectively. Clearly, sample points generated by the constructed features of the 50th generation are more clustered near the feature value of 0. Although ideal experimental results have been achieved on training sets, constructed features misclassify more instances in testing

sets. Compared with the constructed feature for the 50th generation, the constructed feature of the 6th generation has better generalization ability.

The above experiments verify that the single feature construction method (S) can result in overfitting problems. The best features constructed by GP may not achieve the best predictive performance. Increasing the number of GP iterations blindly does not necessarily achieve better classification performance.

5.3. Comparison results

Experimental results using original features and constructed features are shown in Table 3. In the table, ALL denotes the method that use original features, the column “#F” is the average number of features constructed by different feature construction methods, and the column “A ± Std” represents the average and standard deviation of the accuracy of the 30 independent runs. The non-parametric test, Wilcoxon Signed-Rank Test, is used to test whether paired methods have significant differences without assuming they follow a normal distribution. The Wilcoxon Signed-Rank Test between corresponding methods and ALL is performed with significance level of $P\text{-value} = 0.05$, which is denoted on the column “W-test”. The best experimental results of each dataset are marked as bold. Another non-parametric test, Friedman test, is used to test whether there are significant differences among multiple feature construction methods. The Wilcoxon–Nemenyi–McDonald–Thompson Post-hoc test [56] is selected to test whether paired methods among multiple feature construction methods have significant differences. Boxplots of performance differences between paired feature construction methods help us understand which methods are more effective. In Fig. 4, the X-axis denotes performance differences between paired feature construction methods. For example,

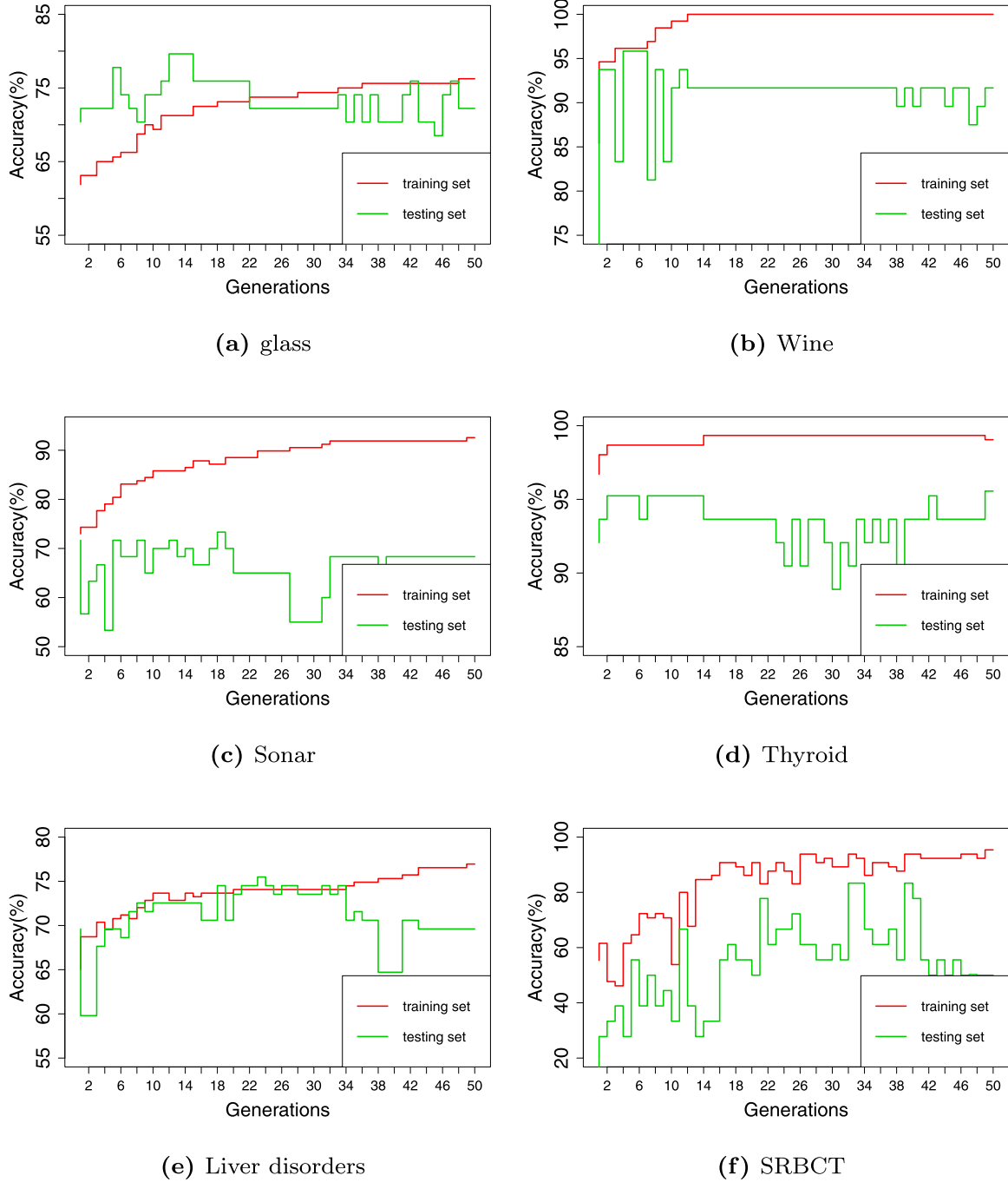


Fig. 2. Overfitting phenomena on feature construction.

MG-S on X -axis shows the performance difference between MG and S. The Post-hoc analysis for Friedman test is marked on boxplots using different colors to show whether two methods are significantly different. If the Post-hoc for the Friedman test finds that $P\text{-value} \leq 0.05$, boxplots are marked green, which shows that performances of paired methods are significantly different. If the Post-hoc for Friedman test finds that $0.05 < P\text{-value} \leq 0.4$, boxplots are marked yellow, which shows that performances of paired methods on most datasets are different.

5.3.1. Comparisons between constructed features with ALL

As shown in Table 3, constructed features can achieve better classification performance than ALL except for Wine, WBC and Lymphoma datasets. Feature construction methods can maintain the classification performance compared with ALL and constructs a much

smaller number of features on the Lymphoma dataset. Due to the influence of overfitting, S does not achieve better classification performance than ALL on glass, Sonar, iris and Liver-disorders datasets. MG and MGMC can achieve significantly better classification performance than ALL on glass, Sonar, iris, Thyroid, Liver-disorders, Wdbc, Colon, Leukemia, SRBCT, MLL and Ovarian datasets. MGMC increases the accuracy by 2.72%, 5.73%, 3.28%, 5.04%, 5.74%, 10.93%, 10.93%, 13.70%, 11.17% on glass, Sonar, Thyroid, Liver-disorders, Colon, Leukemia, SRBCT, MLL and Ovarian datasets respectively than ALL, often constructing a much smaller or similar number of features.

5.3.2. Comparisons between MC and S

The Friedman significance test in Fig. 4 shows that there are significant differences between four feature construction methods with

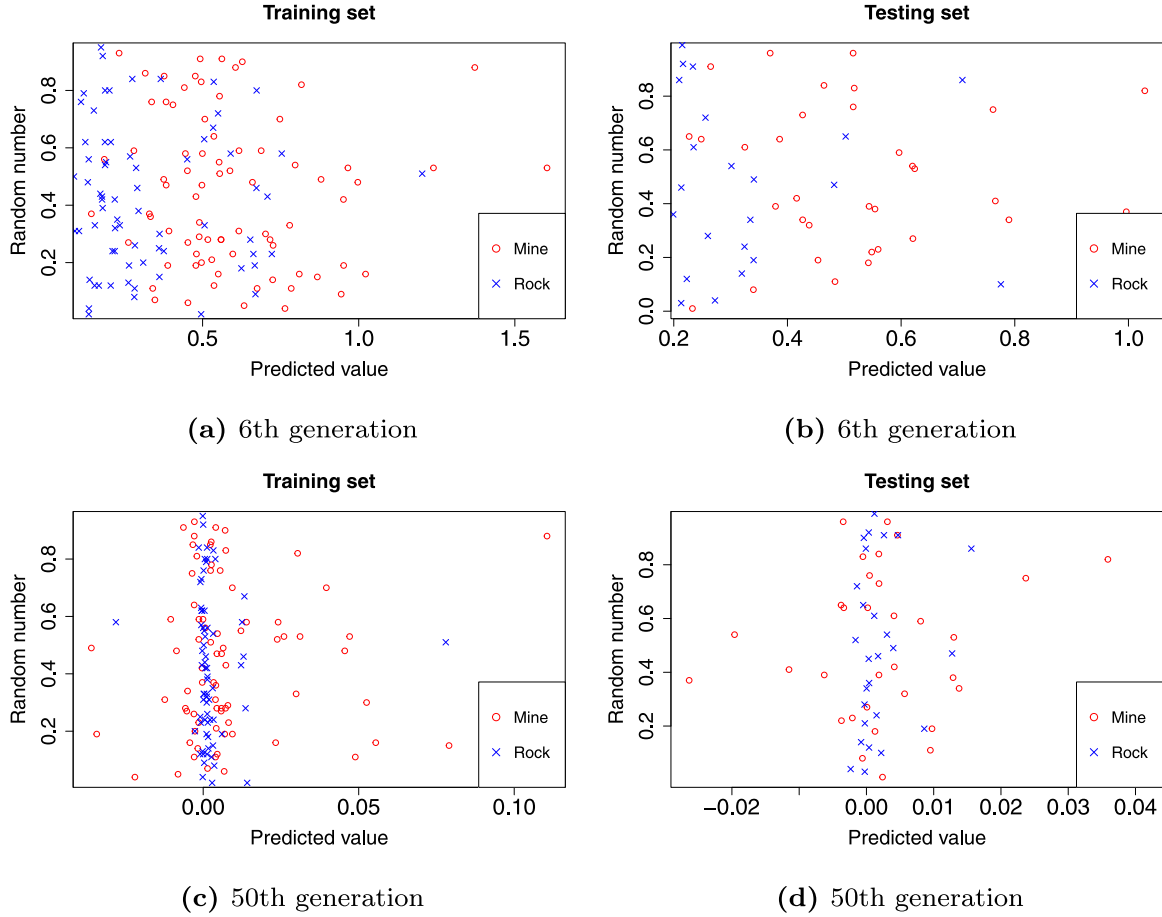


Fig. 3. Some evidences that show overfitting of feature construction on the sonar dataset.

$P\text{-value} = 7.095\text{e-}06$. Though MC cannot achieve significantly better classification performance compared with S, from the experimental results in Table 3, we can see that MC still has advantages over S in most cases.

As shown in Table 3, MC can achieve better classification performance than S on the first eight datasets except for glass and Thyroid datasets. MC achieves equivalent classification performance compared with S on the last six datasets, and the reason may be that there are too few samples and GP can easily match the training set no matter multi-criteria or single-criteria evaluation function is used. The influence of generation on the average accuracy of S and MC on 30 independent runs are shown in Fig. 5. As shown in Fig. 5, MC is superior to S in almost all generations for Wine, Sonar, Liver-disorders, Wdbc and WBC datasets. S has a different degree of overfitting on almost all datasets. However, MC can overcome overfitting problems on Wine, Sonar, Liver-disorders, Wdbc and WBC datasets, and the classification performance does not decrease clearly when the generation exceeds a certain value. Instead, classification performances on Sonar and Liver-disorders datasets increase with the increase of generation.

5.3.3. Comparisons between MG and S, MGMC and MC

The Post-hoc analysis for the Friedman test marked on boxplots in Fig. 4 shows that the performance difference between MG and S is significant with $P\text{-value} = 0.00662$, and the performance difference between MGMC and MC is also significant with $P\text{-value} = 0.00037$. As shown in Table 3, MG can achieve a better classification performance than S on all the datasets and obtain a significantly higher accuracy with 1% to 5% difference than S. Similarly, MGMC can achieve better

classification performance than MC on almost all the datasets. The accuracy of MGMC increases by 3.21%, 4.34%, 2.70% and 3.74% than MC on glass, Sonar, Colon and SRBCT datasets respectively. On other datasets, MGMC is slightly better than MC.

Section 5.2 shows that the single feature construction method on some datasets can result in overfitting problems. To illustrate whether MG and MGMC can avoid overfitting problems, Table 4 shows whether features constructed by MG or MGMC can achieve a better or equivalent classification performance than those produced by any generation of GP using S or MC. We preserved the best constructed features produced by each generation of GP and obtained the average accuracies of those constructed features on 30 independent runs. Suppose average accuracies for all generations are represented as $GENS = \{GEN_1, GEN_2, \dots, GEN_n\}$, where GEN_i is the average accuracy of the constructed feature produced by the i th generation on 30 independent runs. In Table 4, B-S and B-MC denote the maximum of $GENS$ and the elements in $GENS$ are evaluated using S and MC respectively. #GENS denotes the generation that achieves the maximum in $GENS$.

As shown in Table 4, features constructed by MG are better than or equivalent to the feature produced by the best generation of GP using the S method. MG is more effective on glass, Sonar, Colon, Leukemia and SRBCT datasets, and can obtain 1.16%, 2.39%, 0.89%, 0.75% and 0.38% higher accuracy than S in the best generation on those datasets respectively. Similarly, the features constructed by MGMC are better than or equivalent to the feature produced by the best generation of GP using the MC method. The experimental results confirm that multiple features produced by multiple generations can effectively avoid overfitting problems.

Table 3
Classification performance (accuracy) of feature construction methods and ALL.

Dataset	Method	#F	A $\pm$ Std	W-test	Dataset	Method	#F	A $\pm$ Std	W-test
Glass	ALL	9	60.86 $\pm$ 7.02		Wine	ALL	13	95.07 $\pm$ 3.11	
	S	1	61.60 $\pm$ 6.86	=		S	1	90.83 $\pm$ 4.15	-
	MG	12.9	62.78 $\pm$ 6.68	+		MG	6.8	91.46 $\pm$ 3.42	-
	MC	1	60.37 $\pm$ 7.58	=		MC	1	91.81 $\pm$ 3.95	-
	MGMC	13.83	63.58 $\pm$ 7.27	+		MGMC	7.23	92.50 $\pm$ 4.32	-
Sonar	ALL	60	68.83 $\pm$ 6.21		Iris	ALL	4	94.81 $\pm$ 3.31	
	S	1	67.39 $\pm$ 7.92	=		S	1	93.93 $\pm$ 4.44	=
	MG	14	72.50 $\pm$ 6.07	+		MG	3.3	95.70 $\pm$ 3.03	+
	MC	1	70.22 $\pm$ 7.43	+		MC	1	94.52 $\pm$ 3.75	=
	MGMC	14.10	74.56 $\pm$ 4.85	+		MGMC	3.6	95.85 $\pm$ 2.61	+
Thyroid	ALL	5	90.95 $\pm$ 2.90		Liver	ALL	6	58.59 $\pm$ 6.29	
	S	1	94.60 $\pm$ 2.55	+		S	1	60.98 $\pm$ 6.10	=
	MG	5.27	94.76 $\pm$ 2.32	+		MG	17.67	62.81 $\pm$ 6.64	+
	MC	1	94.39 $\pm$ 2.74	+		MC	1	62.78 $\pm$ 6.63	+
	MGMC	5.20	94.73 $\pm$ 2.67	+		MGMC	17.10	63.63 $\pm$ 6.63	+
Wdbc	ALL	30	95.26 $\pm$ 1.49		WBC	ALL	9	96.38 $\pm$ 1.25	
	S	1	95.58 $\pm$ 1.67	=		S	1	95.35 $\pm$ 2.10	-
	MG	7.43	96.03 $\pm$ 1.54	+		MG	8.37	95.80 $\pm$ 1.55	-
	MC	1	95.97 $\pm$ 1.85	+		MC	1	95.76 $\pm$ 1.64	=
	MGMC	6.97	95.65 $\pm$ 1.61	+		MGMC	8.33	95.95 $\pm$ 1.78	=
Colon	ALL	2000	65.74 $\pm$ 8.25		Leukemia	ALL	7129	74.26 $\pm$ 6.17	
	S	1	69.07 $\pm$ 14.68	=		S	1	83.15 $\pm$ 7.38	+
	MG	5.23	71.11 $\pm$ 13.49	+		MG	4.97	83.89 $\pm$ 7.36	+
	MC	1	68.78 $\pm$ 11.78	=		MC	1	83.89 $\pm$ 8.53	+
	MGMC	8.33	71.48 $\pm$ 12.32	+		MGMC	5.90	85.19 $\pm$ 7.08	+
Lymphoma	ALL	4026	80.00 $\pm$ 0.00		SRBCT	ALL	2308	47.22 $\pm$ 12.07	
	S	1	79.11 $\pm$ 2.26	=		S	1	54.81 $\pm$ 11.45	=
	MG	5.70	80.00 $\pm$ 0.00	=		MG	9.90	57.41 $\pm$ 12.53	+
	MC	1	78.22 $\pm$ 4.19	=		MC	1	54.41 $\pm$ 13.89	+
	MGMC	9.90	80.00 $\pm$ 0.00	=		MGMC	22.03	58.15 $\pm$ 14.33	+
MLL	ALL	12582	58.15 $\pm$ 12.65		Ovarian	ALL	15154	85.11 $\pm$ 4.32	
	S	1	68.15 $\pm$ 12.66	+		S	1	95.90 $\pm$ 2.66	+
	MG	3.80	68.89 $\pm$ 11.44	+		MG	2.13	96.24 $\pm$ 2.75	+
	MC	1	70.74 $\pm$ 12.90	+		MC	1	94.74 $\pm$ 3.35	+
	MGMC	6.73	71.85 $\pm$ 10.24	+		MGMC	3.53	96.28 $\pm$ 2.68	+

“+” or “-” in “W-test” means the average accuracy of the feature construction methods is significantly better or worse than ALL. “=” means their performances are similar.

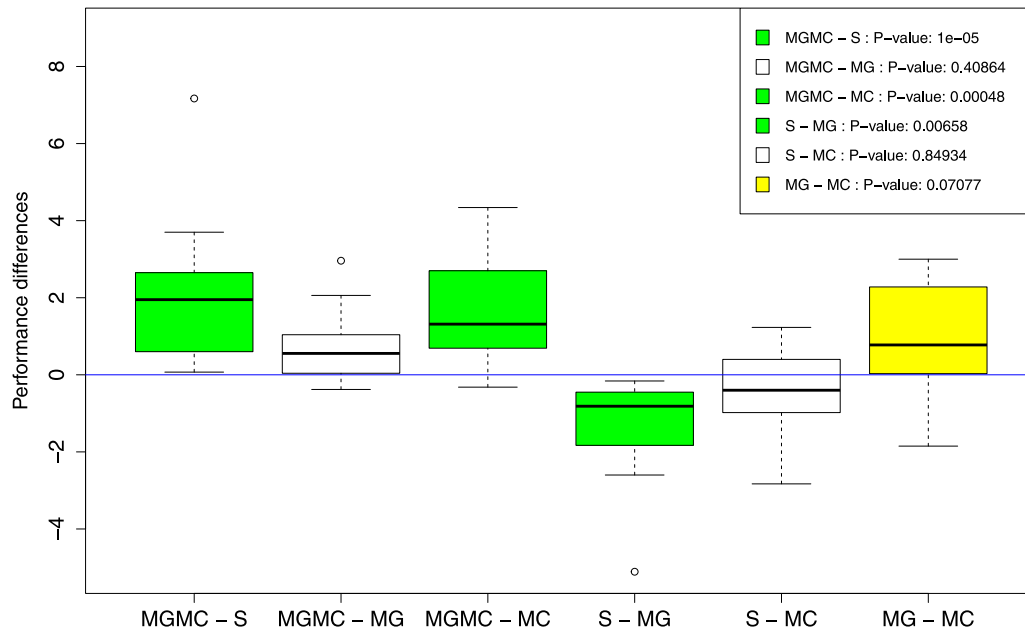
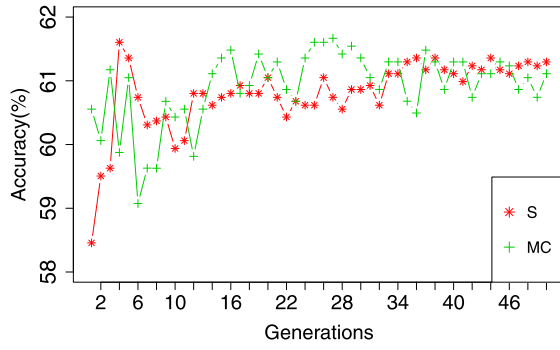


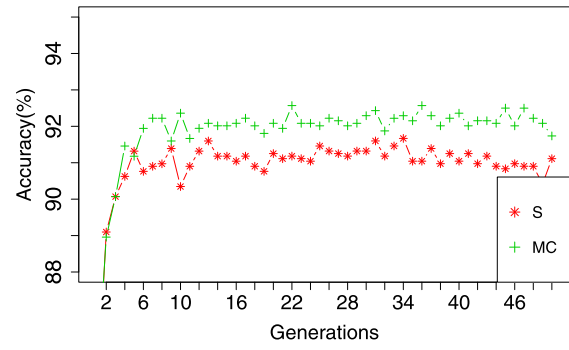
Fig. 4. Boxplots of performance differences between paired feature construction methods marked by Post-hoc analysis for Friedman test (P -value = $7.095e-06$).

As shown in Table 4, #GENS for S is less than 20 on almost all datasets. However, #GENS for MC is less than 30 on most datasets, which is more than that for S. That may be because a multi-criteria

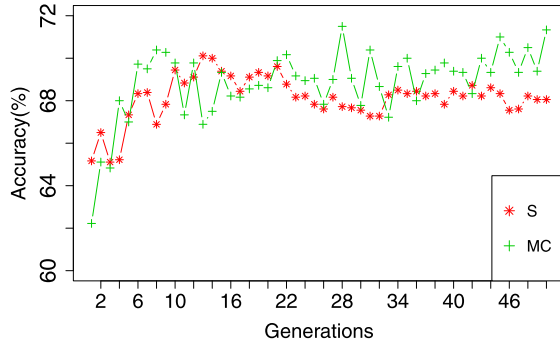
evaluation function is used in MGMC, so the diversity of individuals is increased and the multi-criteria evaluation function makes individuals difficult to converge, which can effectively avoid overfitting problems.



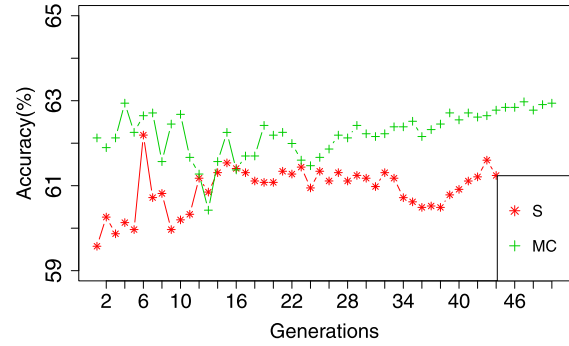
(a) glass



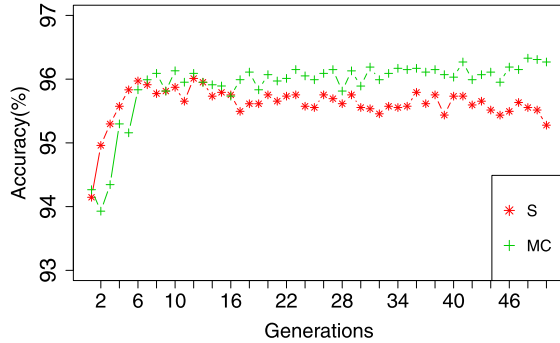
(b) wine



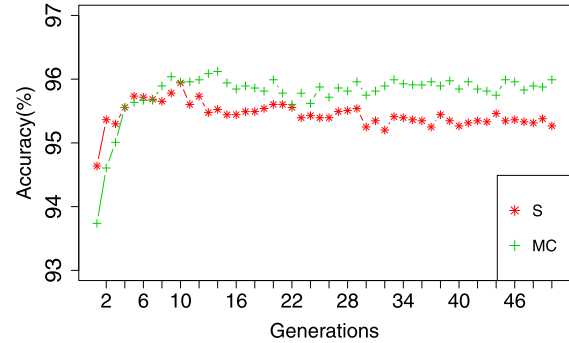
(c) Sonar



(d) Liver-disorders



(e) Wdbc



(f) WBC

Fig. 5. Impacts of generation on S and MC.

5.3.4. Comparisons between MGMC and MG, S, MC

As shown in Table 3, MGMC achieves better classification performance than MG on most datasets. The accuracy of MGMC is 0.80%, 1.04%, 2.06%, 0.82%, 1.30%, 0.74% and 2.96% higher than MG on glass, Wine, Sonar, Liver-disorders, Leukemia, SRBCT and MLL datasets respectively. The experimental results confirm that using a multi-criteria evaluation function can help GP obtain better solutions.

Moreover, from Fig. 4, we can see that MGMC is significantly better than S with $P\text{-value} = 1e-05$. Table 3 shows that MGMC achieves better classification performance than S on almost all the datasets. The above sections have verified that both MG and MC can overcome overfitting problems of S. MGMC is a combination of MG and MC. Experimental results can confirm that MGMC can also effectively avoid overfitting problems of S.

Section 5.3.3 has compared the classification performance of MGMC and MC and has shown that MGMC can achieve better classification performance than MC on almost all the datasets. The above analysis shows that our proposed MGMC can achieve a better classification performance than S, MG and MC.

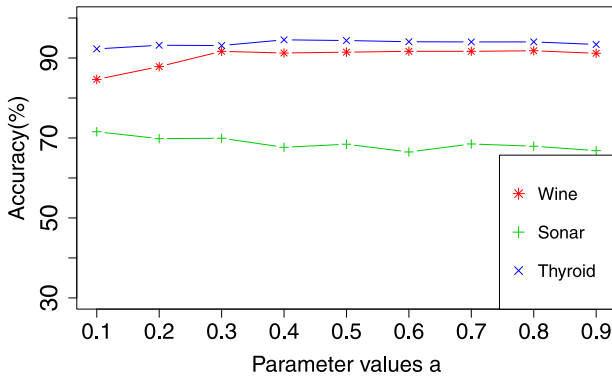
5.4. The impact of parameter α

In order to investigate which parameter value α is the best for each dataset, we randomly selected 30% of a training set as an evaluation set and set the parameter value $\alpha = 0.1, 0.2, \dots, 0.9$ in turn for each dataset to test the experimental results on the evaluation set. The chosen parameter value α for the different datasets is shown in the fifth column in Table 1.

Table 4

Comparisons between MG and S, MGMC and MC in the best generation.

Dataset	MG	B-S	#GENS	MGMC	B-MC	#GENS
Glass	62.78	61.62	4	63.58	61.67	27
Wine	91.46	91.67	13	92.50	92.57	22
Sonar	72.50	70.11	13	74.56	71.50	28
Iris	95.70	95.56	6	95.85	95.85	2
Thyroid disease	94.76	94.87	14	94.73	94.66	20
Liver-disorders	62.81	62.19	6	63.63	62.97	47
Wdbc	96.03	96.01	12	95.65	96.33	48
WBC	95.80	95.94	10	95.95	96.11	14
Colon	71.11	70.22	5	71.48	69.55	9
Leukemia	83.89	83.14	11	85.19	83.52	6
Lymphoma	80.00	80.00	19	80.00	80.00	18
SRBCT	57.41	57.03	13	58.15	55.74	12
MLL	68.89	69.07	8	71.85	71.67	13
Ovarian	96.24	96.15	28	96.28	95.77	24

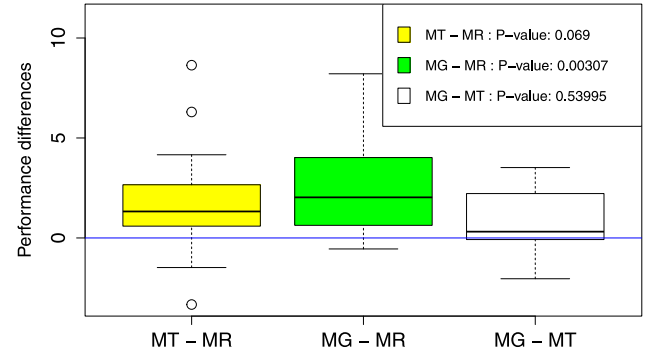
**Fig. 6.** The impact of parameter α on the experimental results for Wine, Sonar and Thyroid datasets.

From the parameter settings in Table 1, we can see that the parameter value $\alpha = 0.8$ is the best for most datasets. Error rate is a major factor when comparing with IGR, so, in general, α should be larger than 0.5. However, a few datasets have exceptions. For example, sonar prefers a parameter value $\alpha = 0.1$, and Thyroid prefers a parameter value $\alpha = 0.4$.

The impact of parameter α on the experimental results for Wine, Sonar and Thyroid datasets is shown in Fig. 6. As shown in Fig. 6, the accuracy decreases with the increase of parameter α on the sonar dataset. For Wine and Thyroid datasets, the accuracy first increases and then decreases with the increase of parameter α , and when the parameter value α reaches a certain value, the accuracy reaches the maximum. It can be seen that the value of the parameter α is closely related to the characteristics of datasets. Setting an appropriate parameter value α can improve the classification effectiveness. It is good to see that the results are not overly sensitive to the parameter α .

5.5. Comparisons between MG and two other multiple feature construction methods

To further verify the effectiveness of our multi-generation feature construction method, two multiple feature construction methods are chosen as benchmarks to compare with. The first one is proposed by Neshatian et al. [8], which uses a fitness function that maximizes the purity of the boundary area of classes and constructs the same number of features as classes. This method needs multiple GP runs and each GP run constructs one feature. This method is named as MR. The second one is proposed by Ma et al. [48], which stores top excellent GP individuals during a single GP run. In this method, the top 10 constructed features are taken into consideration. This method is named as MT. In order to make a fair comparison, the error rate is chosen as the fitness function. The comparison results of MG, MR and MT are shown in Table 5. Boxplots of performance differences between

**Fig. 7.** Boxplots of performance differences between paired feature sets constructed by MR, MT and MG marked by Post-hoc analysis for Friedman test (P -value = 0.003079).

paired feature construction methods marked by the Post-hoc analysis for Friedman test are shown in Fig. 7.

The Friedman Post-hoc tests on three multiple feature construction methods show that P -value = 0.003079, which indicates that three feature construction methods are significantly different. From boxplots in Fig. 7, we can see that MG can achieve significantly better classification performance than MR with P -value = 0.00318. As shown in Fig. 7 and Table 5, though MG cannot achieve significantly better classification performance than MT, MG still achieves better classification performance than MT on most datasets. The experimental results prove further that our proposed multi-generation feature construction method can effectively overcome overfitting problems and can achieve a better classification performance.

Our proposed MG preserves multiple solutions generated by multiple generation of GP, which constructs multiple features in a GP run. MT also constructs multiple features in a GP run. However, this method needs to preserve multiple excellent solutions from all the individuals during a GP run. So compared with MT, MG has lower computational cost. MR needs to run GP multiple times to construct the same number of features as the number of classes. Therefore, the computational cost of MR is higher than that of MT and MG.

5.6. Using other classification algorithms instead of KNN

Neshatian et al. [8] has analyzed that staircase decision boundaries learned by a decision tree fails to obtain ideal performance, because decision tree classifiers are intrinsically incapable of transforming their input space to gain class separability. A good constructed feature can transform the input space into a new space and improve classification results of decision trees [8]. In this section, eight decision tree based classification algorithms including C4.5 [57], BF tree (BFT) [58], NB tree (NBT) [59], Random tree (RT), Random Forest (RaF) [60], Bagging (Bag) [61], Rotation Forest (RoF) [62] and Heterogeneous Oblique Random Forest (HoRaF) [63] are chosen to further verify our feature construction methods. The base learners of Bagging and Rotation Forest are C4.5 decision tree algorithms. The number of trees in RaF and HoRaF is set to 10. LibSVM is selected as the linear classifier of HoRaF. The codes of HoRaF come from <http://www.ntu.edu.sg/home/epnsugan/>. Other classification algorithms are run on the Weka package [64]. The feature construction methods S and MG are used for experimenting and to verify whether feature construction methods on decision trees can achieve similar experimental results compared with KNN. The experimental results are shown in Table 6. In the table, for example, "A-HoRaF" denotes the average accuracy of the 30 independent runs using the HoRaF classification algorithm. The third to the last column is the average accuracy of the corresponding classification algorithms.

From the experimental results in Table 6, we can draw the following conclusions.

Table 5

Classification performance (accuracy) of our proposed MG and two benchmarks (MR [8] and MT [48]).

Dataset	Method	#F	A $\pm$ Std	Dataset	Method	#F	A $\pm$ Std
Glass	MR	6	54.57 $\pm$ 9.61	Wine	MR	3	91.88 $\pm$ 7.28
	MT	10	63.21 $\pm$ 9.27		MT	10	93.06 $\pm$ 3.06
	MG	12.9	62.78 $\pm$ 6.68		MG	6.8	91.46 $\pm$ 3.42
Sonar	MR	2	68.67 $\pm$ 9.83	Iris	MR	3	95.19 $\pm$ 3.09
	MT	10	71.33 $\pm$ 6.52		MT	10	95.78 $\pm$ 2.46
	MG	14	72.50 $\pm$ 6.07		MG	3.3	95.70 $\pm$ 3.03
Thyroid	MR	3	94.13 $\pm$ 2.87	Liver	MR	2	58.79 $\pm$ 1.00
	MT	10	94.13 $\pm$ 2.43		MT	10	60.26 $\pm$ 6.09
	MG	5.27	94.76 $\pm$ 2.32		MG	17.67	62.81 $\pm$ 6.64
Wdbc	MR	2	94.01 $\pm$ 2.93	WBC	MR	2	94.68 $\pm$ 1.32
	MT	10	95.91 $\pm$ 2.07		MT	10	95.30 $\pm$ 1.62
	MG	7.43	96.03 $\pm$ 1.54		MG	8.37	95.80 $\pm$ 1.55
Colon	MR	2	69.07 $\pm$ 11.27	Leukemia	MR	2	79.89 $\pm$ 9.67
	MT	10	67.59 $\pm$ 11.57		MT	10	81.67 $\pm$ 8.98
	MG	5.23	71.11 $\pm$ 13.49		MG	4.97	83.89 $\pm$ 7.36
Lymphoma	MR	3	79.33 $\pm$ 2.00	SRBCT	MR	4	57.96 $\pm$ 12.56
	MT	10	80.00 $\pm$ 0.00		MT	10	54.63 $\pm$ 11.21
	MG	5.70	80.00 $\pm$ 0.00		MG	9.90	57.41 $\pm$ 12.53
MLL	MR	3	64.63 $\pm$ 9.99	Ovarian	MR	2	91.95 $\pm$ 8.50
	MT	10	70.93 $\pm$ 12.40		MT	10	96.11 $\pm$ 2.30
	MG	3.80	68.89 $\pm$ 11.44		MG	2.13	96.24 $\pm$ 2.75

Table 6

Average accuracy on other classification algorithms.

Dataset	Method	A-HoRaF	A-RoF	A-Bag	A-RaF	A-C4.5	A-BFT	A-NBT	A-RT
Glass	ALL	62.96	67.50	65.20	68.77	68.83	60.86	64.75	62.53
	S	57.53	58.62	58.62	54.26	57.41	59.94	55.25	53.70
	MG	63.77	64.18	62.71	61.05	39.26	61.30	58.89	58.70
Wine	ALL	93.44	94.25	88.00	91.60	83.96	82.92	91.67	83.47
	S	91.47	91.09	90.45	90.97	89.86	91.67	92.29	90.56
	MG	92.42	91.95	91.09	91.39	90.55	91.32	92.99	90.83
Sonar	ALL	71.34	76.03	71.00	73.33	68.72	68.28	68.72	66.61
	S	70.87	71.15	72.59	65.83	69.22	70.11	71.50	65.89
	MG	71.10	73.39	72.36	71.22	70.22	71.78	73.28	66.89
Iris	ALL	94.98	95.02	93.10	94.81	93.48	94.81	92.96	94.07
	S	92.59	91.72	91.95	93.78	93.04	94.15	93.93	93.70
	MG	93.85	93.26	93.41	95.41	92.52	95.78	95.11	95.63
Thyroid	ALL	93.76	95.18	90.91	93.17	89.37	89.79	94.66	91.96
	S	94.07	94.14	94.03	93.44	93.23	94.60	94.44	93.39
	MG	94.49	93.81	93.76	93.86	93.86	94.50	94.60	93.70
Liver	ALL	61.04	65.01	63.79	63.95	59.77	60.62	58.46	59.48
	S	58.37	63.89	64.71	58.50	62.91	63.46	64.08	57.94
	MG	61.49	67.00	65.82	61.34	65.95	64.44	67.09	59.08
Wdbc	ALL	95.78	95.48	93.80	94.11	92.22	92.14	92.40	90.87
	S	95.72	95.87	96.00	94.86	95.83	95.93	96.01	94.76
	MG	96.67	96.47	96.35	95.69	95.54	96.15	96.09	94.92
WBC	ALL	96.24	96.80	95.37	95.64	94.19	93.64	95.78	95.78
	S	95.02	94.55	96.30	94.98	96.10	95.72	96.43	96.43
	MG	96.55	95.53	96.49	95.80	95.96	96.19	96.63	96.63
Colon	ALL	74.63	73.18	66.58	64.63	67.78	66.67	63.89	58.33
	S	68.52	66.28	66.86	64.81	68.15	65.74	66.85	63.89
	MG	72.59	70.88	70.50	72.78	71.11	68.33	66.30	66.48
Leukemia	ALL	79.26	82.38	71.26	75.00	67.78	66.30	86.30	64.81
	S	88.70	84.10	86.78	86.48	83.33	86.67	84.44	86.11
	MG	89.04	89.85	88.70	89.44	85.56	87.22	89.44	87.96
Lymphoma	ALL	97.78	91.26	85.06	88.22	84.44	77.56	93.33	86.00
	S	94.22	85.52	85.06	94.67	84.89	90.00	91.11	94.89
	MG	93.78	92.18	88.28	92.89	88.00	84.67	96.44	94.89
SRBCT	ALL	65.19	63.03	57.47	53.52	59.26	55.56	61.11	53.52
	S	69.63	67.43	65.71	66.48	65.37	64.07	63.89	66.48
	MG	75.74	77.20	68.58	72.22	63.15	61.11	71.48	72.22
MLL	ALL	83.89	65.53	50.19	60.93	46.67	38.70	80.19	48.33
	S	76.67	74.90	75.29	70.56	70.74	74.07	62.04	70.19
	MG	75.74	73.95	73.56	73.70	72.59	73.52	68.89	72.96
Ovarian	ALL	97.83	92.89	95.68	87.24	94.89	96.13	96.07	79.11
	S	94.99	95.45	95.23	91.07	92.26	92.52	92.82	91.03
	MG	96.55	96.11	95.89	91.92	92.78	92.95	93.25	91.71

Table 7

Average accuracy of our proposed MG and three baselines (Tran [10], LFS [65] and GPFS) on six microarray datasets.

Dataset	Method	A-HoRaF	A-RoF	A-Bag	A-RaF	A-C4.5	A-BFT	A-NBT	A-RT
Colon	MG	72.59	70.88	70.50	72.78	71.11	68.33	66.30	66.48
	Tran	74.80	72.99	73.56	71.85	71.30	68.33	70.93	68.15
	LFS	80.00	79.31	75.48	76.85	72.96	71.85	78.33	70.37
	GPFS	73.52	75.86	78.74	75.19	76.48	74.63	77.41	71.30
Leukemia	MG	89.04	89.85	88.70	89.44	85.56	87.22	89.44	87.96
	Tran	89.07	91.19	86.78	90.00	86.48	87.04	91.48	87.78
	LFS	92.41	92.15	89.46	90.93	87.59	83.70	94.44	84.63
	GPFS	89.81	89.27	87.93	87.78	84.81	83.89	87.78	83.93
Lymphoma	MG	93.78	92.18	88.28	92.89	88.00	84.67	96.44	94.89
	Tran	98.00	94.71	86.44	94.89	87.33	81.33	96.44	89.11
	LFS	98.44	96.78	87.59	96.67	87.11	78.67	97.78	92.89
	GPFS	97.33	94.71	86.90	94.44	85.89	82.00	97.11	71.11
SRBCT	MG	75.74	77.20	68.58	72.22	63.15	61.11	71.48	72.22
	Tran	77.04	85.44	71.46	78.15	66.67	62.59	83.15	78.15
	LFS	84.07	88.12	71.65	83.33	62.41	65.00	96.30	71.11
	GPFS	78.70	84.10	66.86	76.48	64.81	59.07	82.22	64.07
MLL	MG	75.74	73.95	73.56	73.70	72.59	73.52	68.89	72.96
	Tran	78.33	80.46	74.90	79.81	74.26	72.59	77.78	69.81
	LFS	90.37	88.89	78.74	86.11	67.59	66.11	95.00	71.67
	GPFS	77.59	81.42	74.71	77.04	70.93	66.48	76.30	69.81
Ovarian	MG	96.55	96.11	95.89	91.92	92.78	92.95	93.25	91.71
	Tran	96.10	97.15	95.72	96.18	95.16	95.69	95.64	96.04
	LFS	99.11	98.94	96.37	98.40	95.96	96.84	99.20	95.69
	GPFS	96.96	97.01	94.94	95.24	94.89	94.31	95.69	94.22

(1) Compared with ALL, S and MG can achieve better classification performance in most cases.

As shown in Table 6, in most cases, either S or MG can achieve better classification performance than ALL on decision tree algorithms. Especially in the six high-dimensional microarray datasets, compared with ALL, S or MG can significantly improve classification performance in most cases.

From our experimental results in Table 6, we can see that feature construction methods including S and MG cannot improve the classification performance of RoF and HoRaF. In the first eight datasets, feature construction methods only improve the classification performance on two datasets (Liver-disorders and Wdbc) in RoF. In RoF, Principal Component Analysis (PCA) is applied to feature subsets and transformed original features into principal components (new features) [62] which is similar to our GP-based feature construction. This may provide evidence that feature extraction methods including PCA and GP-based feature construction may improve the classification performance of decision tree algorithms. Feature construction methods can improve the classification performance for only half of the datasets in HoRaF. In HoRaF, an oblique linear hyperplane is applied to each node to overcome the limitations of decision tree algorithms [63]. Feature construction methods have not obtained the same obvious effect as other decision tree algorithms on HoRaF.

(2) Compared with S, MG can achieve better classification performance in most cases, which is consistent with our research objectives.

From the experimental results in Table 6, we can see that MG can achieve significantly better or comparable classification performance than S. In most cases, MG can improve the accuracy of S by 1% to 7%. MG achieves especially better results than S in datasets Sonar, Liver-disorders, Wdbc and Leukemia. These results agree with our previous results obtained using KNN and this verifies that our proposed MG can avoid overfitting of S.

5.7. Further analysis on high-dimensional microarray datasets

We investigated the performance on six high-dimensional microarray datasets to verify the effectiveness of our feature construction methods. These microarray datasets have a large number of redundant features and a small number of samples. There has been existing research that applies feature selection to reduce the dimension of

features and improve the classification performance of these microarray datasets [66,67]. Tran et al. [10] used GP to perform feature construction and feature selection simultaneously on such microarray datasets and achieved good results. In order to compare with these feature selection methods, we introduce three baselines to verify our feature construction methods on the six microarray datasets. The first two employ Linear Forward Selection (LFS) [65] and GP as search algorithms respectively, and the third uses Tran's method. The three algorithms are named as LFS, GPFS and Tran respectively. All the three methods use Correlation [68] to evaluate feature subsets. We will still verify our method on decision tree based classification algorithms.

The experimental results are shown in Table 7. As shown in Table 7, MG can compete with other three baselines on the datasets Leukemia and Lymphoma. On the other four datasets, compared with the three baselines, MG has no advantages. The classification performance of GPFS is worse than that of LFS, which suggests that it may be difficult for GP to search for effective features as terminal nodes from thousands of original features in high-dimensional microarray datasets, which will affect GP's experimental results. Tran has shown the advantage of considering feature selection when constructing features from high-dimensional microarray datasets [10], which further confirms that high-dimensional data will affect the classification performance of GP-based feature construction and feature selection can assist GP in searching for effective original features. Combining feature selection with feature construction methods to improve the classification performances of such small samples and high-dimensional microarray datasets should be investigated further. This is not the focus of this paper.

6. Conclusions

To avoid overfitting problems of the single feature construction approach (S), this paper proposes a Multi-Generation feature construction method (MG) that preserves the individuals produced by multiple generations of GP and a Multi-Criteria feature construction method (MC) that uses a multi-criteria evaluation function to select individuals. Combining MG and MC, another Multi-Generation Multi-Criteria feature construction method (MGMC) is proposed. The experiments on fourteen datasets provide evidences that overfitting problems exist in S. Our proposed MG can achieve better classification performance

than S and overcome overfitting problems of S. Our proposed MC can also improve the classification performance compared with S and can overcome overfitting problems of S in most cases. Our proposed MGMC can obtain better classification performance than S, MG and MC.

Though we have shown evidences that overfitting problems exist in traditional GP based feature construction methods and have proposed MG, MC and MGMC to overcome overfitting problems, there are still problems to be solved. Vanneschi [30] has quantified bloat, overfitting and functional complexity in GP. Further investigations are needed to confirm the relationship between bloat and overfitting, functional complexity and overfitting in GP based feature construction methods. Furthermore, we can try methods to make GP not to memorize various peculiarities of the training set in order to avoid overfitting. Moreover, experiments show that GP's search effect is affected by high-dimensional features. The use of feature selection on high-dimensional datasets to improve the performance of GP-based feature construction needs to be investigated further.

CRedit authorship contribution statement

Jianbin Ma: Conceptualization, Methodology, Investigation, Writing – original draft. **Xiaoying Gao:** Writing – review & editing, Formal analysis. **Ying Li:** Validation, Formal analysis, Visualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work is supported by Hebei Provincial Department of Science and Technology (No. 20327405D) and Hebei Key Laboratory of Agricultural Big Data, China.

We thank Professor P.N. Suganthan for the codes of Heterogeneous Oblivious Random Forest (<http://www.ntu.edu.sg/home/epnsugan/>). We also thank the anonymous reviewers and editors for their careful work and thoughtful suggestions that help to improve this paper substantially.

References

- [1] F. Han, W.T. Chen, Q.H. Ling, H. Han, Multi-objective particle swarm optimization with adaptive strategies for feature selection, *Swarm Evol. Comput.* 62 (2021) 100847.
- [2] Z. Liu, B. Chang, F. Cheng, An interactive filter-wrapper multi-objective evolutionary algorithm for feature selection, *Swarm Evol. Comput.* 65 (2021) 100925.
- [3] K. Neshatian, *Feature Manipulation with Genetic Programming* (Ph.D. thesis), Victoria University of Wellington, 2010.
- [4] J.R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection*, MIT Press, 1992.
- [5] W. Banzhaf, P. Nordin, R.E. Keller, F.D. Francone, Genetic programming: An introduction on the automatic evolution of computer programs and its applications, *J. Combin. Theory* 71 (2) (1998) 130–143.
- [6] H. Majeed, A. Wali, O. Beg, Optimizing Genetic Programming by exploiting semantic impact of sub trees, *Swarm Evol. Comput.* 65 (2021) 100923.
- [7] I. Bakurov, M. Castelli, F. Fontanella, D. Scotto, L. Vanneschi, A novel binary classification approach based on geometric semantic genetic programming, *Swarm Evol. Comput.* 69 (2022) 101028.
- [8] K. Neshatian, M. Zhang, P. Andreae, A filter approach to multiple feature construction for symbolic learning classifiers using genetic programming, *IEEE Trans. Evol. Comput.* 16 (5) (2012) 645–661.
- [9] M.G. Smith, L. Bull, Genetic programming with a genetic algorithm for feature construction and selection, *Genet. Program. Evol. Mach.* 6 (3) (2005) 265–281.
- [10] B. Tran, B. Xue, M. Zhang, Genetic programming for feature construction and selection in classification on high-dimensional data, *Memet. Comput.* 8 (1) (2016) 3–15.
- [11] M. Muharram, G.D. Smith, Evolutionary constructive induction, *IEEE Trans. Knowl. Data Eng.* 17 (11) (2005) 1518–1528.
- [12] F.E.B. Otero, M.M.S. Silva, A.A. Freitas, J.C. Nievola, Genetic programming for attribute construction in data mining, in: *Proceedings of 2003 European Conference on Genetic Programming*, 2003, pp. 384–393.
- [13] S. Ahmed, M. Zhang, L. Peng, B. Xue, Multiple feature construction for effective biomarker identification and classification using genetic programming, in: *Proceedings of the 2014 Conference on Genetic and Evolutionary Computation*, 2014, pp. 249–256.
- [14] A. Subasi, M.I. Gursoy, EEG signal classification using PCA, ICA, LDA and support vector machines, *Expert Syst. Appl.* 37 (12) (2010) 8659–8666.
- [15] B. Du, W. Xiong, J. Wu, L. Zhang, L. Zhang, D. Tao, Stacked convolutional denoising auto-encoders for feature representation, *IEEE Trans. Cybern.* 47 (4) (2017) 1017–1027.
- [16] J. Ma, X. Gao, A filter-based feature construction and feature selection approach for classification using Genetic Programming, *Knowl.-Based Syst.* 196 (2020) 105806.
- [17] J. Loughrey, P. Cunningham, Overfitting in wrapper-based feature subset selection: The harder you try the worse it gets, in: *Proceedings of International Conference on Innovative Techniques and Applications of Artificial Intelligence*, 2004, pp. 33–43.
- [18] T.G. Dietterich, Overfitting and undercomputing in machine learning, *ACM Comput. Surv.* 27 (3) (1995) 326–327.
- [19] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, Dropout: a simple way to prevent neural networks from overfitting, *J. Mach. Learn. Res.* 15 (1) (2014) 1929–1958.
- [20] M. Bramer, Using J-pruning to reduce overfitting in classification trees, *Knowl.-Based Syst.* 15 (5–6) (2002) 301–308.
- [21] J.C. Feng, K.T. Chi, Fundamentals of neural networks, *Technometrics* 37 (4) (2014) 470.
- [22] L. Prechelt, Automatic early stopping using cross validation: quantifying the criteria, *Neural Netw.: Off. J. Int. Neural Netw. Soc.* 11 (4) (1998) 761–767.
- [23] I. Gonçalves, S. Silva, C.M. Fonseca, M. Castelli, Unsure when to stop? Ask your semantic neighbors, in: *Proceedings of 2017 the Genetic and Evolutionary Computation Conference, GECCO*, 2017.
- [24] S.J. Nowlan, G.E. Hinton, Simplifying neural networks by soft weight-sharing, *Neural Comput.* 4 (4) (1992) 473–493.
- [25] D. McNeish, Using lasso for predictor selection and to assuage overfitting: A method long overlooked in behavioral sciences, *Multivar. Behav. Res.* 50 (5) (2015) 471–484.
- [26] D.M. Hawkins, The problem of overfitting, *J. Chem. Inf. Comput. Sci.* 44 (1) (2004) 1–12.
- [27] J. Lever, M. Krzywinski, N. Altman, Model selection and overfitting, *Nature Methods* 13 (9) (2016) 703–704.
- [28] Q. Chen, M. Zhang, B. Xue, Structural risk minimisation-driven genetic programming for enhancing generalisation in symbolic regression, *IEEE Trans. Evol. Comput.* 23 (4) (2019) 703–717.
- [29] K.Y. Chan, C.K. Kwong, T.S. Dillon, Y.C. Tsim, Reducing overfitting in manufacturing process modeling using a backward elimination based genetic programming, *Appl. Soft Comput.* 11 (2) (2011) 1648–1656.
- [30] L. Vanneschi, M. Castelli, S. Silva, Measuring bloat, overfitting and functional complexity in genetic programming, in: *Proceedings of Genetic & Evolutionary Computation Conference*, 2010.
- [31] X. Feng, Y. Liang, X. Shi, D. Xu, X. Wang, R. Guan, Overfitting reduction of text classification based on AdaBELM, *Entropy* 19 (2017) 330.
- [32] A.D. Rocha, T.A. Groen, A.K. Skidmore, R. Darvishzadeh, L. Willems, The Naive Overfitting Index Selection (NOIS): A new method to optimize model complexity for hyperspectral data, *ISPRS J. Photogramm. Remote Sens.* 133 (2017) 61–74.
- [33] Q. Chen, M. Zhang, B. Xue, Feature selection to improve generalisation of genetic programming for high-dimensional symbolic regression, *IEEE Trans. Evol. Comput.* 21 (5) (2017) 792–806.
- [34] R. Liu, D.F. Gillies, Overfitting in linear feature extraction for classification of high-dimensional image data, *Pattern Recognit.* 53 (C) (2016) 73–86.
- [35] J.E. Batista, S. Silva, Improving the detection of burnt areas in remote sensing using hyper-features evolved by M3GP, in: *Proceedings of 2020 IEEE Congress on Evolutionary Computation, CEC*, 2020, pp. 1–5.
- [36] H. Guo, Q. Zhang, A.K. Nandi, Feature extraction and dimensionality reduction by genetic programming based on the Fisher criterion, *Expert Syst.* 25 (5) (2008) 444–459.
- [37] M.A. Muharram, G.D. Smith, Evolutionary feature construction using information gain and gini index, in: *Proceedings of European Conference on Genetic Programming*, 2004, pp. 379–388.
- [38] H. Guo, L.B. Jack, A.K. Nandi, Feature generation using genetic programming with application to fault classification, *IEEE Trans. Syst. Man Cybern. B* 35 (1) (2005) 89–99.
- [39] H. Guo, A.K. Nandi, Breast cancer diagnosis using genetic programming generated feature, *Pattern Recognit.* 39 (5) (2006) 980–987.

- [40] L. Liao, A.K. Pindur, H. Iba, Genetic programming with random binary decomposition for multi-class classification problems, in: *Proceedings of 2021 IEEE Congress on Evolutionary Computation, CEC*, 2021, pp. 564–571.
- [41] H. Firpi, E. Goodman, J. Echaz, On prediction of epileptic seizures by means of genetic programming artificial features, *Ann. Biomed. Eng.* 34 (3) (2006) 515–529.
- [42] K. Krawiec, Generative learning of visual concepts using multiobjective genetic programming, *Pattern Recognit. Lett.* 28 (16) (2007) 2385–2400.
- [43] K. Krawiec, Genetic programming-based construction of features for machine learning and knowledge discovery tasks, *Genet. Program. Evol. Mach.* 3 (4) (2002) 329–343.
- [44] L. Muñoz, S. Silva, L. Trujillo, M3GP–multiclass classification with GP, in: *Proceedings of 2015 European Conference on Genetic Programming (EuroGP)*, 2015, pp. 78–91.
- [45] Y. Lin, B. Bhanu, Evolutionary feature synthesis for object recognition, *IEEE Trans. Syst. Man Cybern. C* 35 (2) (2005) 156–171.
- [46] B. Bhanu, K. Krawiec, Coevolutionary construction of features for transformation of representation in machine learning, in: *Proceedings of Genetic and Evolutionary Computation Conference*, 2002, pp. 249–254.
- [47] K. Krawiec, B. Bhanu, Visual learning by evolutionary and coevolutionary feature synthesis, *IEEE Trans. Evol. Comput.* 11 (5) (2007) 635–650.
- [48] J. Ma, G. Teng, A hybrid multiple feature construction approach using Genetic Programming, *Appl. Soft Comput.* 80 (2019) 687–699.
- [49] B. Tran, M. Zhang, B. Xue, Multiple feature construction in classification on high-dimensional data using GP, in: *Proceedings of 2016 IEEE Symposium Series on Computational Intelligence, SSCI*, 2016, pp. 1–8.
- [50] B. Tran, X. Bing, M. Zhang, Class dependent multiple feature construction using genetic programming for high-dimensional data, in: *Proceedings of Australasian Joint Conference on Artificial Intelligence*, 2017, pp. 182–194.
- [51] D. Dheeru, E. Karra Taniskidou, *UCI machine learning repository*, 2017, URL <http://archive.ics.uci.edu/ml>.
- [52] Z. Zhu, Y.S. Ong, M. Dash, Markov blanket-embedded genetic algorithm for gene selection, *Pattern Recognit.* 40 (11) (2007) 3236–3248.
- [53] B. Xue, M. Zhang, W. Browne, Particle swarm optimization for feature selection in classification: A multi-objective approach, *IEEE Trans. Cybern.* 43 (6) (2013) 1656–1671.
- [54] S. Luke, ECJ then and now, in: *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2017, pp. 1223–1230.
- [55] U. Bhowan, M. Johnston, M. Zhang, Developing new fitness functions in genetic programming for classification with unbalanced data, *IEEE Trans. Syst. Man Cybern. B* 42 (2) (2012) 406–421.
- [56] M. Hollander, D.A. Wolfe, *Nonparametric Statistical Methods*, second ed., Wiley-Interscience, 1999.
- [57] J.R. Quinlan, *C4.5: Programs for Machine Learning*, Morgan Kaufmann, 1993.
- [58] H. Shi, *Best-First Decision Tree Learning* (Master's thesis), The University of Waikato, 2007.
- [59] R. Kohavi, Scaling up the accuracy of Naive-Bayes classifiers: a decision-tree hybrid, in: *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining*, 1996.
- [60] L. Breiman, Random forests, *Mach. Learn.* 45 (1) (2001) 5–32.
- [61] L. Breiman, Bagging predictors" machine learning, *Mach. Learn.* 24 (1996) 123–140.
- [62] J.J. Rodriguez, L.I. Kuncheva, C.J. Alonso, Rotation forest: A new classifier ensemble method, *IEEE Trans. Pattern Anal. Mach. Intell.* 28 (10) (2006) 1619–1630.
- [63] R. Katuwal, P.N. Suganthan, L. Zhang, Heterogeneous oblique random forest, *Pattern Recognit.* 99 (2020) 107078.
- [64] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, I.H. Witten, The WEKA data mining software: an update, *Acml Sigkdd Explor. Newsl.* 11 (1) (2009) 10–18.
- [65] H. Peng, F. Long, C. Ding, Feature selection based on mutual information: Criteria of max-dependency, max-relevance, and min-redundancy, *IEEE Trans. Pattern Anal. Mach. Intell.* 27 (8) (2005) 1226–1238.
- [66] P. Mohapatra, S. Chakravarty, P.K. Dash, Microarray medical data classification using kernel ridge regression and modified cat swarm optimization based gene selection system, *Swarm Evol. Comput.* 28 (2016) 144–160.
- [67] H. Wang, L.J. Tan, B. Niu, Feature selection for classification of microarray gene expression cancers using Bacterial Colony Optimization with multi-dimensional population, *Swarm Evol. Comput.* 48 (2019) 172–181.
- [68] M.A. Hall, L.A. Smith, Feature selection for machine learning: Comparing a correlation-based filter approach to the wrapper, in: *Proceedings of the Twelfth International Florida Artificial Intelligence Research Society Conference*, 1999.